

Black Box Testing Analysis Report

Executive summary

SecureBank was implemented independently in Python and JavaScript and evaluated using 172 shared scenarios per language. All scenarios passed. The combined suite reached 100% Python line and branch coverage, while JavaScript reached 98.60% line and 95.50% branch coverage. These results support the documented simulation contract, but they do not establish production banking readiness.

Technique effectiveness

Equivalence partitioning was the easiest technique to apply initially. It turned large input spaces into recognizable categories: positive amounts, negative amounts, unsupported account types, unregistered payees, and malformed dates. Its most useful contribution was making invalid inputs explicit. For example, numeric strings, booleans, non-finite values, and fractions of a cent must not silently become legitimate transfers. These were potential problems addressed by design, rather than defects discovered in a failing final run.

Boundary value analysis identified the clearest opportunities for financial errors. A Savings balance of 1000.00 incurs a fee, while 1000.01 receives a waiver. Similarly, an account with 100.00 meets the Savings minimum, while 99.99 is suspended. Cumulative transfer boundaries were particularly valuable: two individually acceptable transfers can exceed the daily allowance together. Testing midnight and the first day of the month also exposed the need for an injectable clock instead of tests dependent on the actual date.

Decision tables were the hardest technique to design because conditions overlap and error precedence matters. A transfer can simultaneously lack funds and exceed its allowance. The table requires one documented outcome, not whichever validation happens to execute first. Eight transfer combinations, eleven fee rules, and eight bill-payment combinations made this behavior reviewable. Additional scheduling cases checked affordability again at execution time.

State transition testing gave the strongest confidence in account restrictions. A successful operation is insufficient evidence unless the resulting state is also correct. Sequences covering suspension, recovery, freezing, unfreezing, and closure verified both accepted transitions and rejected operations. During implementation review, the scheduler received an explicit Closed-state guard; ST15 now protects that terminal-state behavior.

Coverage comparison

Independent measurements show how the techniques complement one another:

Technique	Cases	Python line coverage	JavaScript line coverage
Equivalence partitioning	66	75.40%	68.53%
Boundary values	58	78.61%	76.22%
Decision tables	33	78.61%	75.52%
State transitions	15	81.82%	83.21%

These percentages overlap and must not be added. Every technique exercises construction and validation, while state scenarios also traverse several commands in one test. A high line percentage therefore does not mean one technique replaces the others. No technique addresses infrastructure that the simulation does not implement, including persistence, authentication, external settlement, or concurrent requests. The real default clock and JavaScript's unexpected-error propagation also remain outside the shared scenarios.

Real-world application

For a real financial project, decision tables would be my starting priority because stakeholders need to agree on business rules before implementation. Boundary analysis would follow immediately for money, limits, and dates. State sequences would then connect individual operations into customer workflows. Equivalence partitions would keep invalid-input coverage systematic without testing every possible value.

The most useful combination is decision tables plus boundaries: tables identify which conditions matter, and boundary cases challenge their exact comparisons. Combining both with state transitions catches rules whose consequences appear only after multiple actions. Shared cases also help compare implementations, although both languages can still agree on a mistaken expectation.

Recommendations

Next time, I would resolve ambiguous requirements with the product owner before coding, particularly restoration thresholds, suspended-account permissions, and failed scheduled payments. An independent review of expected results would reduce shared-oracle risk. For a larger system, I would add persistence recovery, concurrent-transfer tests, security tests, and property-based checks for money conservation. The main lesson is that coverage measures executed code, while confidence depends on meaningful assertions against an agreed specification.